

Architecture Explanation

Supply Chain & Operations – Incident Management (Multi-Agent System)

1. Problem Overview

Supply chain operations teams regularly receive incident reports related to shipment delays, inventory mismatches, vendor failures, and system downtime. While many incidents are routine and can be resolved automatically, others carry significant business risk due to potential SLA breaches, high operational impact, or uncertainty in the reported information.

The objective of this system is to design a **goal-oriented multi-agent workflow** that:

- Automatically handles low-risk incidents
- Applies policy reasoning to assess business risk
- Escalates incidents that are high-risk, ambiguous, or policy-sensitive
- Produces a clear, structured final decision

This system is implemented using **CrewAI**, with explicit agent roles, sequential handoffs, escalation logic, and guardrails to ensure safe and predictable behavior.

2. Multi-Agent Architecture Overview

The system follows a **sequential, dependency-driven multi-agent design**, where each agent performs a single, well-defined responsibility and passes structured outputs to the next agent.

High-Level Workflow

```
graph TD; A[User Incident] --> B[Incident Classification Agent]; B --> C[Policy Intelligence Agent (Knowledge Base / RAG)]; C --> D[Action Recommendation Agent]; D --> E[ ];
```

Escalation & Priority Decision Agent



Final Structured Decision (JSON)

This architecture ensures:

- Clear separation of concerns
 - Deterministic handoffs
 - Transparent escalation decisions
-

3. Agent Roles and Responsibilities

3.1 Incident Classification Agent

Role:

Interprets the raw incident description and extracts structured signals.

Responsibilities:

- Identify the type of incident (shipment delay, inventory issue, vendor issue, or system issue)
- Assess severity (low / medium / high)
- Estimate a confidence score indicating certainty of classification

Output (Structured JSON):

```
{  
  "incident_type": "...",  
  "severity": "...",  
  "confidence_score": 0.0  
}
```

Design Rationale:

This agent performs **only interpretation**, not policy reasoning or escalation. This separation prevents early bias and keeps downstream decisions grounded in clean inputs.

3.2 Policy Intelligence Agent (Knowledge Base / RAG)

Role:

Applies operational policies, SLA rules, and incident-handling guidelines to assess risk.

Responsibilities:

- Retrieve relevant policy documents from the knowledge base
- Evaluate SLA breach risk
- Determine business impact level
- Provide policy-grounded justification

Output (Structured JSON):

```
{  
  "sla_breach_risk": true,  
  "business_impact": "high",  
  "policy_justification": "..."  
}
```

Design Rationale:

This agent isolates **policy reasoning** from classification. The knowledge base may be backed by embeddings or simulated retrieval, but the architectural pattern mirrors real Retrieval-Augmented Generation (RAG) systems used in enterprise settings.

3.3 Action Recommendation Agent

Role:

Suggests an operational response based on classification and policy analysis.

Responsibilities:

- Recommend whether the incident can be auto-resolved, monitored, or escalated
- Provide a concise rationale

Output (Structured JSON):

```
{  
  "recommended_action": "escalate",  
  "rationale": "..."  
}
```

Design Rationale:

This agent does **not** make the final escalation decision. It acts as an advisory layer, ensuring that escalation authority remains centralized and consistent.

3.4 Escalation & Priority Decision Agent

Role:

Acts as the final decision authority for escalation and priority assignment.

Responsibilities:

- Evaluate all prior agent outputs
- Apply escalation rules and guardrails
- Assign priority level if escalation is required

Output (Final JSON):

```
{  
  "final_decision": "escalate",  
  "priority": "P1",  
  "justification": "..."  
}
```

Design Rationale:

By centralizing escalation logic, the system avoids conflicting decisions and ensures consistent risk handling across all incident types.

4. Task Flow and Agent Handoffs

Each agent depends on structured outputs from the previous agent. No agent operates in isolation.

Handoff Summary

Step	Input	Output
Classification	Raw incident text	Incident type, severity, confidence
Policy Reasoning	Classification output	SLA risk, business impact
Recommendation	Policy + classification	Suggested action
Escalation	All prior outputs	Final decision & priority

This explicit handoff structure satisfies the requirement for **collaborative, non-isolated agents**.

5. Escalation Logic and Guardrails

Escalation decisions are **rule-driven and safety-first**.

An incident is escalated if **any** of the following conditions are met:

- SLA breach risk is **true**
- Business impact is **high**
- Classification confidence is below a defined threshold
- Policy context is insufficient or ambiguous

Why Guardrails Matter

- Prevents silent failures
- Ensures uncertain cases reach human oversight
- Avoids over-automation in sensitive scenarios

When uncertainty exists, the system **defaults to escalation**, prioritizing safety over automation.

6. Sample Inputs and Outputs

Example 1: Routine Delay

Input:

“Shipment to Delhi DC delayed by 12 hours due to heavy rain.”

Output:

```
{  
  "final_decision": "auto_resolve",  
  "priority": "none",  
  "justification": "Delay falls within acceptable SLA threshold with no downstream impact."  
}
```

Example 2: Critical Vendor Failure

Input:

“Vendor X failed to deliver critical components; production at risk.”

Output:

```
{  
  "final_decision": "escalate",  
  "priority": "P1",  
  "justification": "Critical vendor failure with high business impact and production risk."  
}
```

Example 3: Ambiguous Inventory Issue

Input:

“Something seems wrong with stock numbers in storage area.”

Output:

```
{  
  "final_decision": "escalate",  
  "priority": "P2",  
  "justification": "Low classification confidence requires manual review."  
}
```

7. Design Rationale and Key Decisions

Key Design Choices

- **Sequential agent workflow** for clarity and traceability
- **Single-responsibility agents** to avoid role overlap
- **Structured JSON outputs** for deterministic handoffs
- **Explicit escalation rules** rather than vague heuristics
- **Guardrail-first design** to handle uncertainty safely

Why This Architecture Works

- Mirrors real enterprise incident-management workflows
 - Scales easily with additional policies or agents
 - Easy to audit, debug, and extend
 - Balances automation with human oversight
-

8. Conclusion

This multi-agent system demonstrates a robust, realistic approach to supply chain incident management. By combining structured agent collaboration, policy-aware reasoning, and explicit escalation logic, the system achieves reliable automation while maintaining safety and control.

The architecture satisfies all mandatory project requirements and reflects best practices used in real-world AI-assisted operations systems.